

Transformer-Based Language Models

Amir-Hossein Mohammand-Pour & Michael Franke

Preliminaries: tokens, vocabulary, language model

Let $\mathcal{V} = \{t_1, \dots, t_{n_{\mathcal{V}}}\}$ be the *vocabulary*, a finite set of *tokens* of cardinality $n_{\mathcal{V}}$. The vocabulary is ordered by the bijective *indexing function* $\text{Ind}: \mathcal{V} \rightarrow \{1, \dots, n_{\mathcal{V}}\}$, so that $\text{Ind}(t)$ is the (*token*) *index* of t . A *token sequence* of length $n \in \mathbb{N}$ is a vector $\mathbf{t} \in \mathcal{V}^n$ of n tokens from the vocabulary. The set of all finite token sequences forms the *language* $\mathcal{L} = \{\mathbf{t} \in \mathcal{V}^n \mid n \in \mathbb{N}\}$.

Aside. As will become clear below, nothing in this definition or the way in which the transformer-based model is defined actually hinges on the nature of $\mathcal{V}$. To stay clear of over-interpretation of a model’s abilities (e.g., regarding language understanding), it helps to think of a language model as operating solely on the *indices* of tokens from the vocabulary: an autoregressive language model is designed to predict the next integer in a sequence of integers from a finite range $1, \dots, n_{\mathcal{V}}$. Indeed, we should probably better speak of a “finite discrete sequence model,” was it not so much of an economic non-starter unwieldy mouthful. ◀

A (*generative / decoder*) *language model* is a function $\mathcal{M}_{\theta}: X \rightarrow \Delta(\mathcal{L})$, parameterized on a set of model parameter values $\theta \in \Theta$, which maps some input set X onto a probability distribution over all (finite) sequences of tokens. The input set X could be some kind of trigger, like a picture or a piece of text, for which a token sequence needs to be produced. The set X could also be empty, in which case the language model just tries to capture the unconditional occurrence probability of particular sequences of tokens (e.g., the sequences of words that speakers of a particular language would be likely to produce, given the constraints of their language’s grammar and whatever governs when speakers want to communicate particular pieces of information). In the following, we just speak of a language model or transformer model, leaving implicit that we focus entirely on generative / decoder LMs.

Aside. The term “language model” can also be used in a wider sense, beyond models that *predict* or *produce* token sequences, namely for models that merely *process* (i.e., take as input) a token sequence, e.g., to return a compact representation (an embedding) of an input sequence for multiple downstream tasks. The transformer architecture we introduce here can also be used for this purpose, but we focus here on the case of sequence prediction.

For practical reasons, rather than defining a full holistic probability over $\mathcal{L}$, the probability of a token sequence is usually calculated as a product of conditional probabilities of individual tokens given some other parts of the sequence. There are two prominent kinds of such factorizations: next-token and missing-token language models. An *autoregressive (next-token prediction) language model* is a function $\mathcal{M}_{\theta}: \mathcal{L} \rightarrow \Delta(\mathcal{V})$ which maps a token sequence onto a *next-token probability*

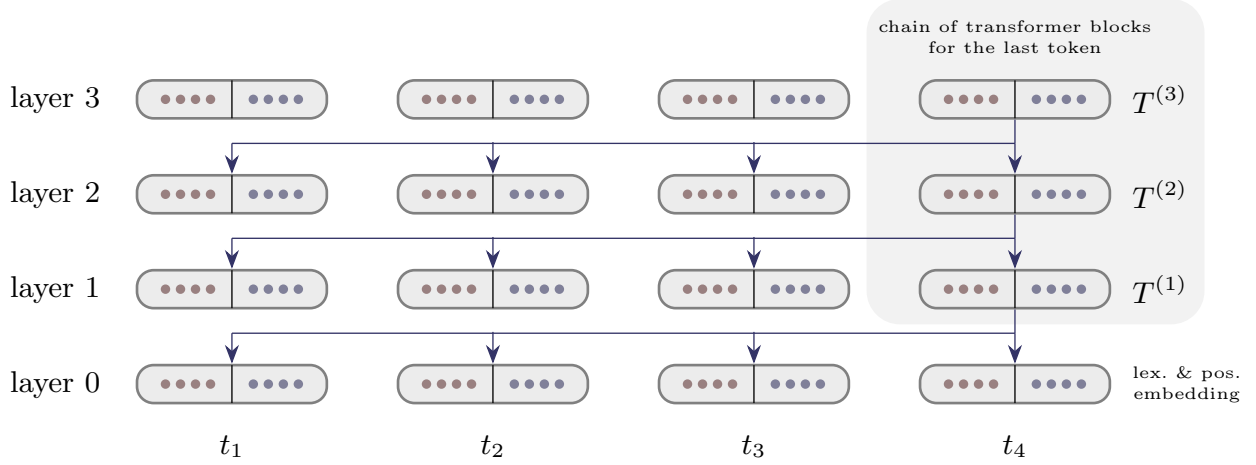


Figure 1: General transformer architecture for a model with $n_{\mathcal{L}} = 3$ layers, $n_{\mathcal{H}} = 2$ attention heads and a model size of $d_{\mathcal{M}} = 8$, when applied to an input of length $N = 4$. The (vertical) chain of transformer blocks that computes the embedding for the last, 4th token is highlighted. At each layer ℓ , the transformer block $T^{(\ell)}$ computes a new embedding based on the embeddings of all tokens $t_1, \dots, t_4$ at the previous layer.

distribution. We write $P_{\mathcal{M}}(t \mid \mathbf{t}; \theta)$ or, omitting the model’s parameters, $P_{\mathcal{M}}(t \mid \mathbf{t})$ for the model’s prediction of the next-token probability for token t . In contrast, a *masked (missing-token prediction) language model* is a function that predicts selected tokens inside of a given sequence of tokens. The next-token prediction LMs can be considered a special case of the missing-token language models. For ease of exposition of the main architecture we focus on the former.

Aside. Masked language models are also called “bidirectional models” because they allow to use information from the left and right part of a sequence (from a given relevant token position i). The class of masked language models is more fuzzy than that of autoregressive models. Masked language models can be trained to actually predict a missing word, a fill sequence of tokens, or to identify which words in a given sequence should be replaced. This is also why we focus most heavily on the autoregressive case, but our definitions are general enough to also capture stereotypical cases of masked language modeling.

General transformer architecture

For an input token sequence $\mathbf{t}$ of length n_C (*context size*), the transformer model processes the sequence in parallel across all positions using a chain—stack—of $n_{\mathcal{L}}$ transformer blocks. We refer to the ℓ^{th} element in the chain as the ℓ^{th} *layer*. Importantly, the ℓ^{th} layer is, essentially, the same (i.e., using the same parameters) for every token position i in the sequence. The larger the number $n_{\mathcal{L}}$ of layers, the more iterative information-processing steps the transformer architecture takes. Large language models will tend to have more layers and tend to be more powerful as a result.

Transformer blocks & embeddings. Each transformer block is, essentially, a mapping from one or several *embeddings* to another. An embedding $\mathbf{x} \in \mathbb{R}^{d_M}$ is an d_M -dimensional vector. Embeddings may be referred to alternatively as latent representations, hidden representations, latent vectors or similar. We call d_M the *dimensionality of the embedding space*, the size of the latent representations or similar. So, as each transformer block outputs an embedding of size d_M , we may think of the transformer architecture as a way to compute embeddings of size d_M for each input token at each layer.

The transformer block $T^{(\ell)}$ for the ℓ^{th} layer is a function $T^{(\ell)}: \{1, \dots, n_C\} \times (\mathbb{R}^{d_M})^{n_C} \rightarrow \mathbb{R}^{d_M}$ which maps a the i^{th} input position of context C ($1 \leq i \leq n_C$) and a sequence of context embeddings to a single embedding as output, such that:

$$T^{(\ell)}: \left(\underbrace{i}_{\text{index of token in sequence}}, \underbrace{\langle \mathbf{x}_1^{(\ell-1)}, \dots, \mathbf{x}_{n_C}^{(\ell-1)} \rangle}_{\text{embeddings of tokens 1 through } n_C \text{ at previous layer}} \right) \mapsto \underbrace{\mathbf{x}_i^{(\ell)}}_{\text{embedding of } i \text{ at current layer}}$$

For the first layer, $\ell = 1$, the transformer block takes as input the zero-layer token embeddings defined below.

Aside. The above definition of a transformer block, where the input spans the full context size, is most general and faithful to how common implementations proceed. We will use masking (below) to make sure that next-token and missing-token prediction LMs have “access” to only the legitimately available information.

Zero-layer, initial token embeddings. The zero-layer, initial token embedding, $\mathbf{x}_i^{(0)} \in \mathbb{R}^{d_M}$, of input token i is computed as:

$$\mathbf{x}_i^{(0)} = F(\mathbf{W}_{E_{\text{Ind}(i)}}, \text{PosEmbed}(i)),$$

where $\mathbf{W}_E \in \mathbb{R}^{n_V, d_M}$ is the *embedding matrix*, the values of which are part of the model’s trainable parameters, $\text{PosEmbed}(i) \in \mathbb{R}^{d_M}$ is a *positional embedding function* applied to the index of i in the input sequence, and $F: \mathbb{R}^{d_M} \times \mathbb{R}^{d_M} \rightarrow \mathbb{R}^{d_M}$ implements some way of combining the information from the (non-positional, fixed) embedding matrix with the positional information.

Aside. A prominent positional encoding function uses trigonometric functions, where the j^{th} element of the positional encoding of token i is calculated as:

$$\text{PosEmbed}(i)_j = \begin{cases} \sin\left(i \left(n^{\frac{i}{d_M}}\right)^{-1}\right) & \text{if } i \text{ is even, and} \\ \cos\left(i \left(n^{\frac{i-1}{d_M}}\right)^{-1}\right) & \text{otherwise.} \end{cases}$$

A common choice for F in this positional encoding scheme is just element-wise addition. ◀

Unembedding and token predictions. The predictions for a token at position i in the sequence are derived from the embedding $\mathbf{x}_i^{(n_L)}$ for the input token $i = n_C$ at the final layer $\ell = n_L$. We apply the model’s *unembedding matrix* $\mathbf{W}_U \in \mathbb{R}^{n_V, d_M}$, which consists entirely of free model parameters, to $\mathbf{x}_i^{(n_L)}$ to obtain a vector of *output logits* $\text{logits}_i(\mathbf{t}) \in \mathbb{R}^{n_V}$:

$$\text{logits}_i(\mathbf{t}) = \mathbf{W}_U \mathbf{x}_i^{(n_L)}$$

Logits are transformed into probabilities using the softmax function, so that the model’s predicted probability of the next-token becomes:

$$P_M(t \mid \mathbf{t}, i) = \frac{\exp(\text{logits}_i(\mathbf{t})_{\text{Ind}(t)})}{\sum_{j=1}^{n_V} \exp(\text{logits}_i(\mathbf{t})_j)}$$

Aside. The predictions for token position i is a prediction about the *next* token $i+1$ for (autoregressive) next-token prediction models, but it is a prediction about the *current* token i for missing-token prediction models.

While (autoregressive) language models are usually defined and trained with this next-token probability distribution in mind, it is nevertheless possible and usual in applications to use different *decoding schemes*, i.e., ways of mapping the logits, as defined above, onto probabilities, such as modulating the temperature of the softmax function or using some version of beam-search decoding (e.g., Ippolito et al., 2019; Zarrieß, Voigt, and Schütz, 2021). ◀

Anatomy of a transformer block

Following the high-level definition of a transformer block, as introduced above, a transformer for layer ℓ at position i is a function $T^{(\ell)}: \langle i, \langle \mathbf{x}_1^{(\ell-1)}, \dots, \mathbf{x}_{n_C}^{(\ell-1)} \rangle \rangle \mapsto \mathbf{x}_i^{(\ell)}$ that maps a token index and a sequence of embeddings to a new embedding. This function is commonly composed of four main components: (i) an attention module, (ii) a feed-forward neural network, (iii) a layer-normalization step, and (iv) the addition of so-called residual connections. While there are degrees of variation in the internal design of the attention models, we follow the original design of Vaswani et al. (2017), in which these components are chained in the way pictured in Figure 2, which results in a definition of the transformer block as follows:

$$\begin{aligned} \mathbf{x}_i^{(\ell-1,*)} &= \text{LayerNorm}^{(\ell,1)} \left(\mathbf{x}_i^{(\ell-1)} + \text{Attention}^{(\ell)} \left(i, \mathbf{x}_1^{(\ell-1)}, \dots, \mathbf{x}_{n_C}^{(\ell-1)} \right) \right) \\ \mathbf{x}_i^{(\ell)} &= \text{LayerNorm}^{(\ell,2)} \left(\mathbf{x}_i^{(\ell-1,*)} + \text{FFN}^{(\ell)} \left(\mathbf{x}_i^{(\ell-1,*)} \right) \right) \end{aligned}$$

Notice that this representation does not include a name for the residual connection function, but they are represented nonetheless. When we add a *residual connection* to an operation or function F applied to input x , we compute the function $G(x): x \mapsto x + F(x)$, which is practically useful for efficient training (He et al., 2016). Let’s unpack the remaining three components in reverse order of difficulty.

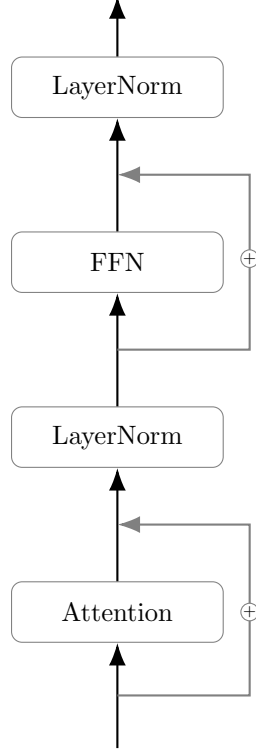


Figure 2: Anatomy of a transformer block. A single transformer block sequentially applies three main functions: (i) attention, (ii) layer normalization, and (iii) a non-linear mapping implemented as a feed-forward network. On top of that, it uses residual connections, as shown by the gray arrows.

Layer normalization. The *layer-normalization function* $\text{LayerNorm}: \mathbb{R}^{d_M} \rightarrow \mathbb{R}^{d_M}$ is included for training efficiency (Ba, Kiros, and Hinton, 2016). It applies first a standardization and then a linear transformation to the input embedding:

$$\text{LayerNorm}(\mathbf{x})^{(\ell,k)} = \boldsymbol{\gamma}^{(\ell,k)} \odot \frac{\mathbf{x} - \mu(\mathbf{x})}{\sigma(\mathbf{x})} + \boldsymbol{\beta}^{(\ell,k)},$$

where:

- $\mu(\mathbf{x}) = \frac{1}{d} \sum_{i=1}^{d_M} x_i$ is the mean of the values in the embedding,
- $\sigma(\mathbf{x}) = \sqrt{\frac{1}{d} \sum_{i=1}^{d_M} (x_i - \mu(\mathbf{x}))^2 + \epsilon}$ is the standard deviation with a small constant $\epsilon > 0$ for numerical stability,
- $\boldsymbol{\gamma}^{(\ell,k)}, \boldsymbol{\beta}^{(\ell,k)} \in \mathbb{R}^{d_M}$ are learned parameters (scale and shift), one for each layer ℓ and occurrence k of the layer-normalization operation in the transformer block, and
- $\odot$ denotes elementwise multiplication.

Aside. The expression “layer normalization” is established but unfortunate in this context —stemming from an older loose term, “batch normalization”. The operation performed is strictly speaking a case of “standardization” and it is not applied to a layer, but an embedding.

Feed-forward network. The *feed-forward network* is just a standard multi-layer feed-forward network instantiating a mapping of the form $\text{FFN}: \mathbb{R}^{d_M} \rightarrow \mathbb{R}^{d_M}$. Commonly, this component is a two-layer network, i.e., a feed-forward neural network with one hidden layer, which is the smallest non-linear feed-forward architecture. For this case, we would define the function $\text{FFN}(\cdot)$ as:

$$\text{FFN}^{(\ell)}(\mathbf{x}_i^{(\ell,*)}) = \mathbf{W}_{out}^{(\ell)} \left(\phi \left(\mathbf{W}_{in}^{(\ell)} \mathbf{x}_i^{(\ell,*)} \right) + \boldsymbol{\beta}^{(in,\ell)} \right) + \boldsymbol{\beta}^{(out,\ell)},$$

where $\mathbf{W}_{in}^{(\ell)} \in \mathbb{R}^{d_{\text{FFN}} \times d_M}$ is the mapping from the input layer to the hidden layer, which has size d_{FFN} (usually about twice the size of the model's embeddings d_M), $\mathbf{W}_{out}^{(\ell)} \in \mathbb{R}^{d_M \times d_{\text{FFN}}}$ is the mapping from hidden layer to output layer, $\boldsymbol{\beta}^{(in,\ell)}$ and $\boldsymbol{\beta}^{(out,\ell)}$ are the offsets of the feed-forward transformations, and ϕ is some (non-linear) activation function. All of $\mathbf{W}_{in}^{(\ell)}, \mathbf{W}_{out}^{(\ell)}, \boldsymbol{\beta}^{(in,\ell)}, \boldsymbol{\beta}^{(out,\ell)}$ are trainable parameters.

Aside. These parameters for the FFN function of the layer ℓ transformer block are the same for each token in the input sequence. They are also not shared across layers, but independently optimized for each layer ℓ . ◀

Attention. The *attention module* is the heart and, as such, the most complex piece of the transformer architecture. From the outside, it is a function of the same type as the transformer block itself: for token t_i at layer ℓ , the attention module is a function $\text{Attention}^{(\ell)}: \{1, \dots, n_C\} \times (\mathbb{R}^{d_M})^{n_C} \rightarrow \mathbb{R}^{d_M}$ that maps a token index and a sequence of n_C embeddings onto another embedding. Internally, this mapping is computed, usually, in terms of n_H *attention heads* operating in parallel.

$$\begin{aligned} \text{Attention}^{(\ell)}(i, \mathbf{x}_1^{(\ell-1)}, \dots, \mathbf{x}_{n_C}^{(\ell-1)}) &= \mathbf{W}_O \mathbf{z}, \text{ where} \\ \mathbf{z} &= \underbrace{\text{AH}^{\ell,1}(i, \mathbf{x}_1^{(\ell-1)}, \dots, \mathbf{x}_{n_C}^{(\ell-1)}) \parallel \dots \parallel \text{AH}^{\ell,n_H}(i, \mathbf{x}_1^{(\ell-1)}, \dots, \mathbf{x}_{n_C}^{(\ell-1)})}_{\text{concatenation of outputs of } n_H \text{ attention heads}} \end{aligned}$$

Here, $\mathbf{W}_O \in \mathbb{R}^{d_M \times n_H d_h}$ is an output mapping, where d_h is the dimension of the output of a single attention head (see below). The output mapping is applied to $\mathbf{z} \in \mathbb{R}^{n_H d_h}$, the concatenation of the output of n_H attention heads.

Each attention head $h \in \{1, \dots, n_H\}$ is a function $\text{AH}^{\ell,h}: \{1, \dots, n_C\} \times (\mathbb{R}^{d_M})^{n_C} \rightarrow \mathbb{R}^{d_h}$, which computes a weighted sum of so-called *value vectors*, one for each token $t_1, \dots, t_{n_C}$:

$$\text{AH}^{\ell,h}(i, \mathbf{x}_1^{(\ell-1)}, \dots, \mathbf{x}_{n_C}^{(\ell-1)}) = \sum_{j=1}^{n_C} \text{Weight}^{(\ell,h)}(\mathbf{x}_i^{(\ell-1)}, \mathbf{x}_j^{(\ell-1)}) \text{Value}^{(\ell,h)}(\mathbf{x}_j^{(\ell-1)})$$

Aside. The weight and value functions use parameters specific to layer ℓ , whence the superscripts for the functions in the equation above, but take as input embeddings from the previous layer (see Figure 1). The weight function does, but the value function does *not* depend on the embedding of the current token i , whence the different number of arguments.

In practice, attention heads might only take a subset of the input embeddings into account, e.g., by partitioning the input space into n_H equally sized slices of the input token sequence. ◀

The *value function* $\text{Value}: \mathbb{R}^{d_M} \rightarrow \mathbb{R}^{d_V}$ maps an embedding of size d_M to a value vector of size d_V . It is implemented via multiplication with a matrix $\mathbf{W}_V^{(\ell,h)} \in \mathbb{R}^{d_V \times d_M}$:

$$\text{Value}^{(\ell,h)}(\mathbf{x}) = \mathbf{W}_V^{(\ell,h)} \mathbf{x}$$

The *weight function* $\text{Weight}: \mathbb{R}^{d_M} \times \mathbb{R}^{d_M} \rightarrow \mathbb{R}^{d_V}$ maps a pair of embeddings of size d_M to a real-valued weight, expressing how much the computation of the next embedding for token i “cares about” or “should attend to” the information in the embedding of token j . To compute the weight for tokens i and j , we first look at the values of two more auxiliary functions, the *key function*:

$$\text{Key}^{(\ell,h)}(\mathbf{x}) = \mathbf{W}_K^{(\ell,h)} \mathbf{x},$$

and the *query function*:

$$\text{Query}^{(\ell,h)}(\mathbf{x}) = \mathbf{W}_Q^{(\ell,h)} \mathbf{x},$$

both of which implement functions from $\mathbb{R}^{d_M}$ to $\mathbb{R}^{d_k}$ for some dimensionality d_k of the key and query vectors. Next, based on the key and query functions, we define a *score function* which takes two embeddings and returns a number that represents (in non-normalized form) how much the information from position j matters to the computation of i :

$$\text{Score}^{(\ell,h)}(\mathbf{x}_i, \mathbf{x}_j) = \begin{cases} -\infty & \text{if token } j \text{ is masked for } i \\ \text{Query}^{(\ell,h)}(\mathbf{x}_i) \cdot \text{Key}^{(\ell,h)}(\mathbf{x}_j) & \text{otherwise.} \end{cases}$$

If token j is accessible from position j (not masked for i), the score is computed via a measure of vector similarity, such as the dot-product in the equation above. If token j is not accessible from the position i (e.g., when $j > i$ for autoregressive (left-to-right) language modeling), the score function returns negative infinity, which in connection with the softmax operation below will result in all information from j being ignored for the computation at position i . (For missing-token prediction, we might say that j is masked for i whenever $j = i$, but more complex cases can be constructed as well.) Finally, using the score function, we define the *attention weight function* via softmax (where we assume that $\exp(-\infty) = 0$):

$$\text{Weight}^{(\ell,h)}(\mathbf{x}_i, \mathbf{x}_j) = \frac{\exp(d_k^{-1/2} \text{Score}^{(\ell,h)}(\mathbf{x}_i, \mathbf{x}_j))}{\sum_{j'=1}^C \exp(d_k^{-1/2} \text{Score}^{(\ell,h)}(\mathbf{x}_i, \mathbf{x}_{j'}))}$$

Aside. For an autoregressive (left-to-right, decoder-only) language model, we could, in principle, also compute the embedding $\mathbf{x}_i^{(\ell)}$ at token i for layer ℓ by attending to the embeddings of all previous tokens *at the same layer* ℓ . However, there are two reasons why this is not common practice. First, when using bi-directional encoding (attending to both previous and subsequent tokens) same-layer attention would be impossible due to circularity. So, for parallelism, autoregressive models may want to do the same. More importantly, however, for performance efficiency, we want to compute all tokens in parallel at each layer (batch them together). That’s why attention is defined by “looking back” at the previous layer. ◀

References

- Ba, Jimmy Lei, Jamie Ryan Kiros, and Geoffrey E. Hinton (2016). *Layer Normalization*. arXiv: [1607.06450](https://arxiv.org/abs/1607.06450) [stat.ML].
- He, Kaiming et al. (2016). “Deep Residual Learning for Image Recognition”. In: *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. IEEE.
- Ippolito, Daphne et al. (2019). “Comparison of Diverse Decoding Methods from Conditional Language Models”. In: *Proceedings of the 57th Annual Meeting of the Association for Computational Linguistics*. Association for Computational Linguistics, pp. 3752–3762.
- Vaswani, Ashish et al. (2017). “Attention is All you Need”. In: *Advances in Neural Information Processing Systems*. Ed. by I. Guyon et al. Vol. 30.
- Zarrieß, Sina, Henrik Voigt, and Simeon Schütz (2021). “Decoding Methods in Neural Language Generation: A Survey”. In: *Information* 12.355, pp. 1–39.